ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТ 1 ТАЙЛАН

Үйлдлийн систем (F.CSM302)
2026-2027 оны хичээлийн жилийн намар

Бие даалтын ажлын нэр: Dining-Philosophers-Problem
Хичээл заасан багш: Г.Ганбат
Багийн дугаар: 4-2-11
Багийн гишүүд: Оюутан 1: А.Аминхишиг B242270011
Оюутан 2: Д.Гантогтох B242270139
Оюутан 3: Т.Буяндэлгэр B222270052
Оюутан 4: Б.Билэгбарьмид B241940060
Оюутан 5: Г.Ёндонжунай B231940060
Оюутан 6: З.Билгүүдэй B200950011

Улаанбаатар
2026

ХУРААНГУЙ

Энэхүү бие даалтын ажлаар үйлдлийн системийн олон процесст орчин дахь синхрончлол, дундын нөөцийн өрсөлдөөн болон гацалт (Deadlock)-аас сэргийлэх механизмыг харуулдаг классик "Dining Philosophers Problem"-ийг сурах бичгийн Project 3-ийн дагуу POSIX mutex lock болон condition variable ашиглан C хэл дээр хэрэгжүүлж, түүнийг визуаль байдлаар харуулах вэбэд суурилсан интерактив симуляторыг хөгжүүлэн, Testing & QA хэсгийг гүйцэтгэв.

- **Асуудал:** Үндсэн C симуляц ба веб интерфейс хооронд холбогдох үед порт зөрчигдөх (5500 Live Server vs 8000 Python Backend) болон CORS алдаа үүсэх, мөн C binary файл буруу замд байршсанаас сервер зогсох асуудлууд тулгарсан.
- **Шийдэл:** Python HTTP/SSE (Server-Sent Events) Bridge серверийг статик веб файлуудыг давхар үйлчилдэг нэгдмэл архитектуртай болгож, C binary-г хөрвүүлэн репозиторийн root директорт байршуулав.
- **Туршилт ба гол үр дүн:** Автомат тестийн 92 шалгалт бүгд давж, философичдын тоо N=2–10 хүрээнд болон 1 мс-ийн стресс хугацаанд ч зэргэлдээ хоёр философич зэрэг хооллосон тохиолдол үүссэнгүй (viol=0), систем гацсангүй. Хүрээнээс гадуурх оролтыг C, сервер, UI гурван түвшинд татгалзаж ойлгомжтой мэдээлэл өгч, server.py зогсоход UI дээр 'ERROR' төлөв харуулдаг болсон.

ГАРЧИГ

1. Зургийн жагсаалт 3
2. УДИРТГАЛ 4
3. СУРАХ БИЧГИЙН ШААРДЛАГА 4
4. ОНОЛЫН ҮНДЭС 4
5. СИСТЕМИЙН ЗОХИОМЖ 5
6. ХЭРЭГЖИЛТ 6
7. ТУРШИЛТ БА UI ДҮРСЛЭЛ 7
8. ТУРШИЛТЫН АРГА 8
9. ҮР ДҮН 9
10. ХЭЛЭЛЦҮҮЛЭГ 12
11. ДҮГНЭЛТ 13
12. АШИГЛАСАН МАТЕРИАЛ 13

1. ЗУРГИЙН ЖАГСААЛТ

Зураг 1 Системийн компонент ба өгөгдлийн урсгал 6
Зураг 2 UI-ийн үндсэн дүрслэл (N=5; Thinking, Hungry, Eating төлөв ба сэрээний өнгө) 7
Зураг 3 N=10 үеийн UI дүрслэл 10
Зураг 4 Буруу оролтод UI-ийн 'ERROR' анхааруулга (жишээ нь N=-3) 10
Зураг 5 server.py зогсоход UI-ийн 'ERROR' төлөв 11
Зураг 6 Автомат тестийн терминал гаралт (92 checks, ALL TESTS PASSED) 12

2. УДИРТГАЛ

Орчин үеийн үйлдлийн системүүд нь олон процессыг нэгэн зэрэг (concurrently) ажиллуулах чадвартай байдаг. Гэвч олон процесс дундын нөөцийг (shared memory, files, hardware) нэгэн зэрэг эзэмшихээр өрсөлдөх үед гацалт (Deadlock) болон нөөцийн хомсдол (Starvation) үүсэх эрсдэлтэй байдаг. Энэхүү төслийн зорилго нь Эдсгер Дейкстрагийн санал болгосон 'Хооллох философичдын асуудал'-ыг C хэлний параллель боловсруулалт болон вэб технологид суурилуулан симуляц хийж, багийн хэмжээнд Manual-ын дагуу хялбар ажиллуулж, өөр өгөгдөл, параметрээр давтан турших боломжтой, алдаанд тэсвэртэй симуляторыг хөгжүүлж, чанарын баталгаажуулалт (Testing & QA) хийхэд оршино.

3. СУРАХ БИЧГИЙН ШААРДЛАГА

Абрахам Сильбершацын 'Operating System Concepts' [1] сурах бичгийн 7-р бүлгийн 'Programming Projects' хэсгийн Project 3 — The Dining-Philosophers Problem-д заасан үндсэн шаардлагууд:

- 0–4 дугаартай таван философич бүр тусдаа thread (POSIX Threads - pthreads) байж, thinking болон eating үйлдлийг thread-ийн sleep-ээр (сурах бичигт 1–3 секундийн санамсаргүй хугацаа) дүрслэх.
- Сэрээ авах `pickup_forks(int philosopher_number)` болон буцаах `return_forks(int philosopher_number)` функцийг POSIX mutex lock болон condition variable ашиглан, сурах бичгийн 7.1.3-ийн monitor шийдэл (Figure 7.7)-ийн дагуу хэрэгжүүлэх.
- Философичдын төлвийн шилжилтийг (Thinking, Hungry, Eating) бодит цагт мэдээлэх; хөрш хоёр философич хэзээ ч зэрэг хооллохгүй, Deadlock үүсэхгүй байх нөхцөлийг хангах.
- БД1-ийн нэмэлт шаардлага: N болон Think/Eat хугацааг өөрчилж дахин ажиллуулах, буруу оролтыг шалгаж ойлгомжтой мэдээлэх, UI дүрслэлийг бодит ажиллагаатай холбох.

4. ОНОЛЫН ҮНДЭС

Үйлдлийн системийн процесст параллелизм дэх Deadlock үүсэх 4 үндсэн шаардлага:

1. **Mutual Exclusion:** Нэг сэрээг зөвхөн нэг философич нэг цагт ашиглана.
2. **Hold and Wait:** Философич нэг сэрээгээ барьчихсан атлаа нөгөө сэрээг хүлээнэ.
3. **No Preemption:** Бусад философичийн барьсан сэрээг хүчээр дээрэмдэхгүй.
4. **Circular Wait:** Философич бүр дараагийнхаа философичийн сэрээг хүлээх цагираг сүлжээ үүсэх.

Сурах бичгийн 7.1.3 хэсгийн monitor шийдэлд философич бүр THINKING, HUNGRY, EATING гэсэн гурван төлөвтэй байна. Философич зөвхөн зүүн, баруун хөрш хоёр нь EATING биш үед л хооллож эхэлнэ. Хоёр сэрээг нэг дор (атомаар) авдаг тул Hold and Wait нөхцөл эвдэгдэж Deadlock үүсэхгүй. Манай симуляцд нэг mutex (pthread_mutex_t) нь төлвийн массивыг хамгаалж, философич бүрт нэг condition variable (pthread_cond_t) харгалзан, хөрш нь сэрээгээ тавихад л хүлээж буй философичийг сэрээнэ.

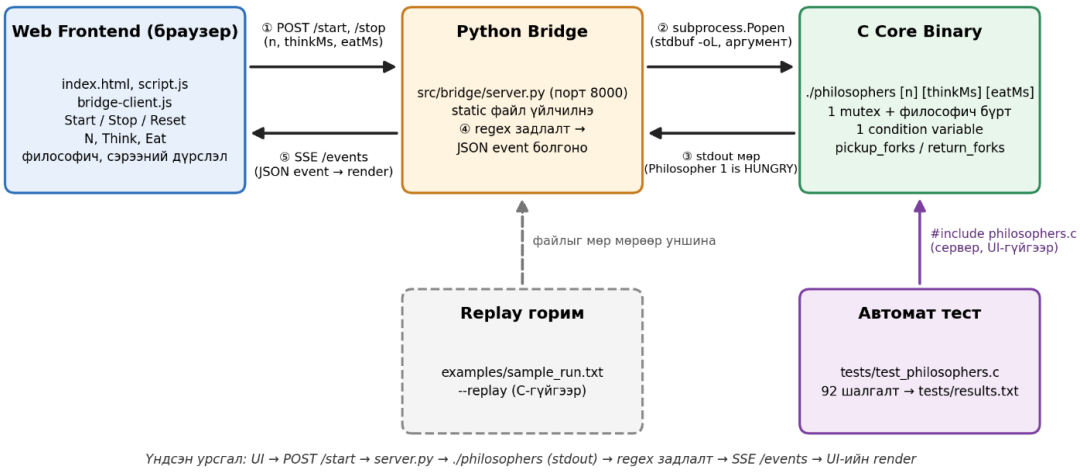
Starvation (нөөцийн хомсдол): Энэ шийдэл нь хөрш хоёр философич зэрэг хооллохгүй байхыг баталгаажуулдаг боловч философич бүрт хооллох боломж заавал олгогдохыг (fairness) онолын хувьд баталгаажуулдаггүй. Иймд туршилтад философич бүрийн хооллосон тоо (meals min..max) болон HUNGRY-аас EATING болтол хамгийн их хүлээлтийн хугацааг (max_wait_ms) хэмжиж fairness-ийг шалгасан.

5. СИСТЕМИЙН ЗОХИОМЖ

Системийн зохиомж нь 3 давхаргат архитектураас бүрдэнэ:

- **C Core Binary (./philosophers, src/philosophers.c):** POSIX thread, нэг mutex ба философич бүрт нэг condition variable ашиглан monitor шийдлийг хэрэгжүүлж, stdout-оор төлвийн мөрүүдийг хэвлэнэ. `./philosophers [n] [thinkMs] [eatMs]` гэсэн аргумент авна.
- **Python Bridge Server (src/bridge/server.py):** Subprocess ашиглан C binary-г ажиллуулж, stdout-ын мөр бүрийг regex-ээр JSON event болгон Server-Sent Events (SSE, `/events`) замаар веб рүү дамжуулна. `POST /start`, `POST /stop`-оор удирдаж, static frontend файлуудыг мөн 8000 порт дээр өгнө. `--replay` горимд examples/sample_run.txt-ийг C binary-гүйгээр дахин тоглуулна.
- **Web Frontend (src/frontend: index.html, script.js, bridge-client.js, style.css):** `EventSource`-оор `/events`-ийг сонсож, `fetch`-ээр `/start`, `/stop` хүсэлт илгээн UI-г бодит цагт шинэчилнэ.

Зураг 1-д системийн гурван үндсэн давхарга болон тэдгээрийн хоорондын өгөгдлийн урсгалыг харуулав. Хэрэглэгч Web Frontend дээр Start товчийг дарахад браузер Python Bridge сервер рүү N, Think, Eat утгуудтай POST /start хүсэлт илгээнэ (①). Сервер subprocess.Popen-оор C Core Binary-г ажиллуулж (②), С программ философичдын төлвийн өөрчлөлтийг stdout-д мөр мөрөөр (жишээ нь "Philosopher 1 is HUNGRY") хэвлэнэ (③). Сервер эдгээр мөрийг regex-ээр задлан JSON event болгож (④), SSE /events сувгаар браузерт дамжуулснаар UI бодит цагт шинэчлэгдэнэ (⑤). Доод хэсгийн тасархай хүрээтэй блокууд нь туслах хэсэг бөгөөд Replay горим нь C программгүйгээр хадгалсан гаралтын файлыг дахин тоглуулдаг, Автомат тест нь philosophers.c-г шууд #include хийж сервер, UI-гүйгээр 92 шалгалт ажиллуулдаг.



Зураг 1 Системийн компонент ба өгөгдлийн урсгал

6. ХЭРЭГЖИЛТ

Гол шийдлүүд болон ашигласан технологиуд:

- **Backend C код:** `pthread_create()`, `pthread_mutex_lock()`, `pthread_mutex_unlock()`, `pthread_cond_wait()`, `pthread_cond_signal()` функцүүдийг ашигласан. `pickup_forks()`, `return_forks()`, `test()` функцүүд Figure 7.7-ийн дагуу хэрэгжсэн.
- **Оролтын шалгалт:** `./philosophers [n] [thinkMs] [eatMs]` — n нь 2–10, хугацаа 100–10000 мс. `parse_arg()` функц буруу утгад ойлгомжтой алдааны мессеж хэвлэж 1 кодоор гарна.
- **Python Bridge:** `subprocess.Popen()` ашиглан C binary-г `stdbuf -oL` тушаалаар мөр мөрөөр нь баферлан ажиллуулж, stdout-ийг уншиж UI руу дамжуулсан (`stdbuf` нь Linux/WSL орчинд байдаг).
- **Тохиргоо дамжуулах:** Серверийг `--pass-args` сонголттойгоор эхлүүлснээр UI-ийн N, Think, Eat утга C программд аргументаар дамжина. Энэ сонголтгүй үед сервер N=5-аас бусдыг татгалзаж, C программ өөрийн default утгаар (Think=1000 мс, Eat=2000 мс) ажиллана.
- **Bridge Client (bridge-client.js):** Алдааны менежментийг хариуцаж, `showError()` функцээр серверийн холболт тасарвал (EventSource `onerror`) эсвэл сервер хүсэлтийг татгалзвал UI дээр 'ERROR' төлөв болон тайлбарыг мэдээлнэ.
- **Автомат тест:** tests/test_philosophers.c нь src/philosophers.c-г `#include` хийж, `parse_arg()`, `test()`/`return_forks()`-ийн дүрэм, N=2–10 хүрээний scaling болон стресс тестийг ажиллуулж, үр дүнг tests/results.txt-д хадгална.

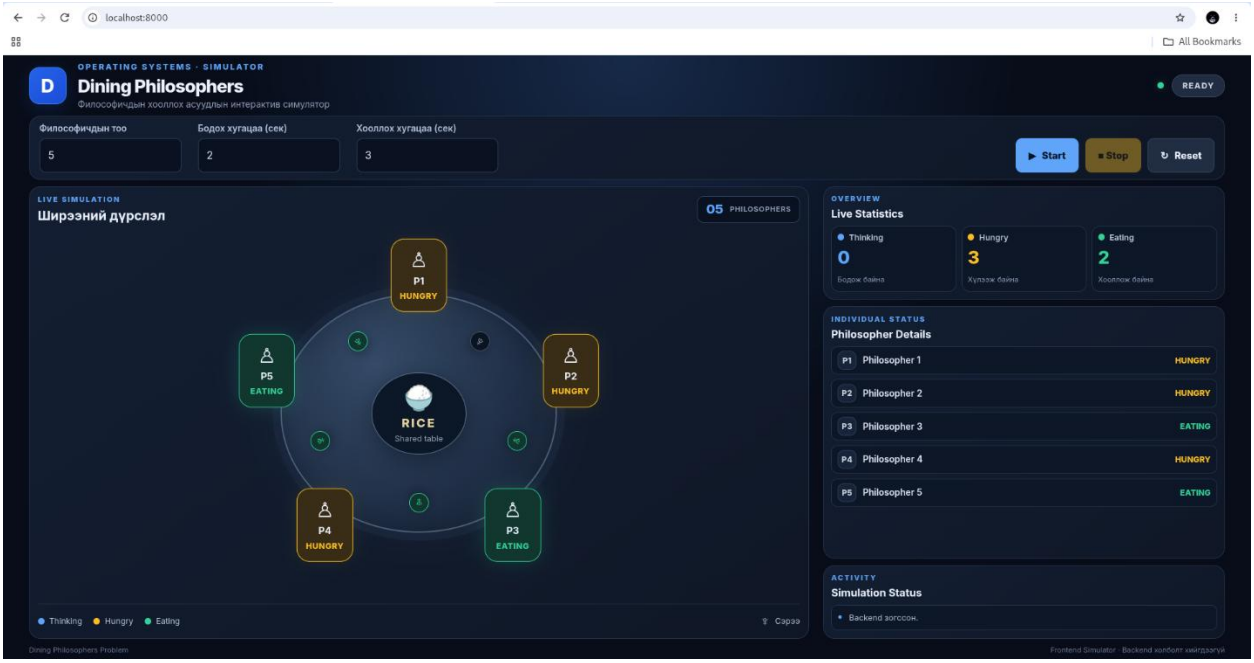
- **Ашигласан сан, бэлэн код, AI:** Гадны сан ашиглаагүй (C стандарт сан ба POSIX Threads, Python стандарт сан, цэвэр HTML/CSS/JavaScript).

AI хэрэгслийн ашиглалт: Багийн гишүүд өөрсдийн хариуцсан кодын хэсгүүдийг (C core, Python bridge, frontend, тест) хөгжүүлэхдээ AI хэрэгсэл ашигласан. Ашигласан хэрэгсэл: Gemini, Claude.

7. ТУРШИЛТ БА UI ДҮРСЛЭЛ

Туршилт болон UI дүрслэлийн хүрээнд шалгасан зүйлс:

- **N-ийг өөрчлөх:** N=2 (хамгийн бага), N=5 (стандарт), N=10 (хамгийн их) утгаар философичдын тоог UI дээр өөрчилж, дугуй ширээн дээрх философич, сэрээний дүрсүүд өөрчлөгдөж байгааг ажиглана. 2–10-аас гадуурх утгыг татгалзана.
- **Хугацааны тохиргоо:** UI дээр Think/Eat хугацааг 1–10 секундийн (бүхэл) хооронд, сервер ба C түвшинд 100–10000 мс хооронд өөрчилж шалгана. Эдгээр утга C программд хүрэхийн тулд серверийг `--pass-args`-тай эхлүүлсэн байх ёстой.
- **Ажигласан зүйл:** Философичдын 'Thinking', 'Hungry', 'Eating' төлвүүдийн тоолуур болон өнгөн заалтууд бодит цагт үнэн зөв шинэчлэгдэж байв.
- **UI-ийн удирдлага:** Start — симуляц эхлүүлнэ; Stop — backend-ийг зогсооно; Reset — backend-ийг зогсоож философичдыг эхний төлөвт буцаана. N, Think, Eat талбарыг өөрчилбөл автоматаар reset хийгдэнэ.



Зураг 2 UI-ийн үндсэн дүрслэл (N=5; Thinking, Hungry, Eating төлөв ба сэрээний өнгө)

Зураг 2 нь N=5 философичтой үеийн UI-ийн үндсэн дэлгэцийг харуулна. Дээд хэсэгт философичдын тоо, бодох болон хооллох хугацааны оролтын талбар, Start, Stop, Reset товчнууд байрлана. Зүүн талын дугуй ширээн дээр философич бүр (P1–P5) одоогийн төлвөөрөө өнгөлөгдөнө: цэнхэр нь Thinking (бодож байна), шар нь Hungry (сэрээ хүлээж байна), ногоон нь Eating (хооллож байна). Философичдын хооронд сэрээ байрлах бөгөөд ногоон сэрээг философич барьж байгааг, бараан сэрээ чөлөөтэй байгааг илтгэнэ. Баруун талд төлөв тус бүрийн тоолуур (энэ агшинд Thinking 0, Hungry 3, Eating 2) болон философич тус бүрийн төлвийн жагсаалт харагдана. Хоолж буй P3 ба P5 нь хоорондоо хөрш биш тул хөрш хоёр философич зэрэг хооллохгүй гэсэн нөхцөл биелж байгааг энэ зураг харуулж байна.

8. ТУРШИЛТЫН АРГА

Туршилт хийсэн орчин ба аргууд:

- **Орчин:** Linux Debian / WSL2, GCC 12.2, Python 3.11, Google Chrome Browser.
- **Захын утгын тест (Boundary Testing):** N-д 1, 0, -3, 20 болон тоо биш утга ('abc'), Think/Eat-д 50 мс, 20000 мс оруулахад C программ ойлгомжтой алдааны мессеж хэвлэж 1 кодоор гарна; сервер HTTP 400 ба `error` талбартай хариу өгнө; UI 'ERROR' төлөв ба тайлбар харуулна.
- **Сүлжээний ба сөнөөх тест (Negative Testing):** server.py-г зориудаар зогсооход UI 'ERROR' төлөвт шилжинэ (EventSource `onerror`). Харин зөвхөн C процессыг kill хийвэл сервер `stopped` event илгээж, UI 'READY' төлөв ба "Backend зогссон." мэдээлэл харуулна.
- **Автомат тест (tests/test_philosophers.c):** `gcc tests/test_philosophers.c -o test_ph -pthread` ба `./test_ph` тушаалаар ажиллуулна. Агуулга: `parse_arg()`, буруу аргументтай `main()`, `test()`/`return_forks()`-ийн дүрэм, N=2,3,5,8,10 дээрх 3 секундийн scaling хүснэгт, Think/Eat=1 мс-ийн стресс, N=7-ийн 15 удаагийн давталт. Хэмжсэн үзүүлэлт: meals (нийт хооллолт), viol (зэргэлдээ хоёр философич зэрэг хооллосон тоо), max_eat (зэрэг хооллосон дээд тоо), meals min..max (философич бүрийн хооллолтын зөрүү), max_wait_ms (HUNGRY-аас EATING болтол хамгийн их хүлээлт).
- **Хүлээгдэж буй үр дүн:** viol=0, max_eat ≤ ⌊N/2⌋, max_wait нь ажиллуулсан хугацаанаас бага (philosopher өлсөж үлдээгүй), бүх шалгалт PASS.

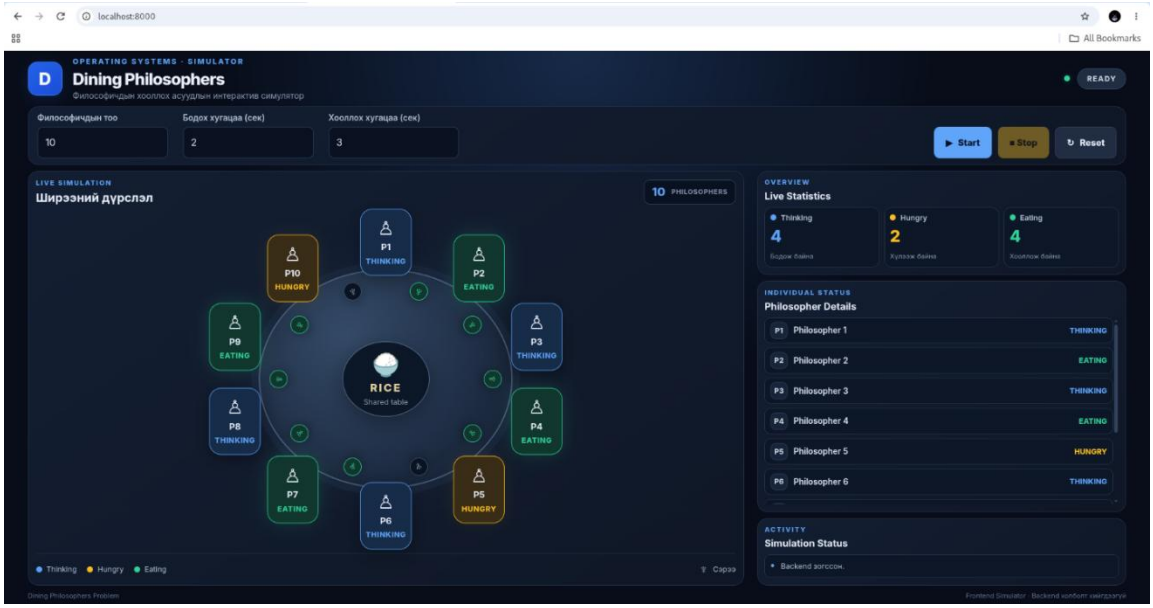
9. ҮР ДҮН

Туршилтын үр дүнгүүдийг нэгтгэсэн хүснэгт болон төлөвүүд:

Хүснэгт 1 QA ба Симуляцйн туршилтын нэгдсэн үр дүн

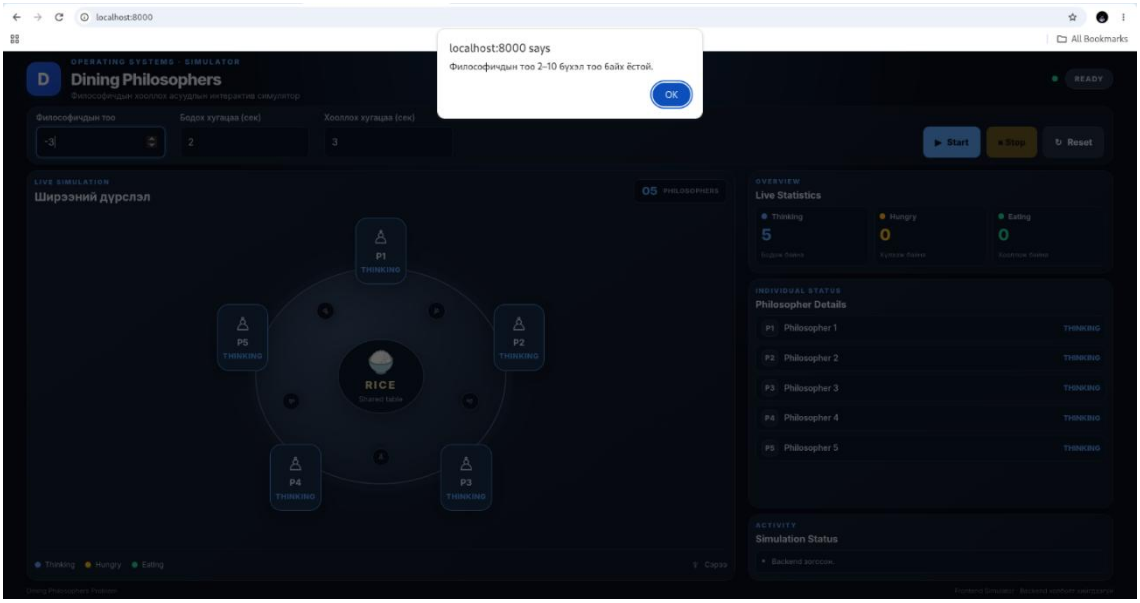
Туршилтын тохиолдол	**Оролтын өгөгдөл**	**Хүлээгдэж буй үр дүн**	**Бодит үр дүн & Төлөв**
Хамгийн бага N	N=2, Think=Eat=20 мс, 3 с	Зэрэг хооллогч ≤1, viol=0	Амжилттай (meals=98, viol=0, max_eat=1, max_wait=28 мс)
Стандарт ажиллагаа	N=5, Think=Eat=20 мс, 3 с	max_eat ≤2, viol=0, гацалтгүй	Амжилттай (meals=198, viol=0, max_eat=2, meals 34..42, max_wait=149 мс)
Хамгийн их N	N=10, Think=Eat=20 мс, 3 с	max_eat ≤5, viol=0	Амжилттай (meals=475, viol=0, max_eat=5, meals 47..48, max_wait=98 мс)
Стресс	N=10, Think=Eat=1 мс, 3 с	viol=0, гацалтгүй	Амжилттай (meals=986, viol=0, max_eat=5, max_wait=39 мс)
Давтан ажиллуулах	N=7, Think=1 мс, Eat=5 мс, 15 удаа	Бүх ажиллуулалт асуудалгүй	Амжилттай (асуудалтай ажиллуулалт 0/15)
Буруу оролт (C)	n=1, 0, -3, 20, 'abc'; thinkMs=50; eatMs=20000	Алдааны мессеж, гарах код 1	Амжилттай (бүгд татгалзагдсан)
Буруу оролт (сервер)	POST /start: n=-3; default горимд n=8	HTTP 400 ба тайлбар	Амжилттай (мессеж: "Философичдын тоо 2–10 байх ёстой."; N≠5-д "--pass-args" сануулга)
Тохиргоо дамжилт	--pass-args, Think=3000 мс, N=8	Эхний HUNGRY ≈3000 мс-д	Амжилттай (t=3006 мс). --pass-args-гүй үед t=1006 мс (C-ийн default Think=1000 мс)
Backend тасалдал	server.py зогсоох	UI дээр 'ERROR' харуулах	Амжилттай (bridge-client.js барьсан)
C процесс тасалдал	C процессыг kill хийх	Сервер stopped event илгээх	Амжилттай (UI: READY, "Backend зогссон.")

Тайлбар: тоон утгууд (meals, max_wait) ажиллуулалт бүрд өөр байж болох бөгөөд үр дүнгийн эх хувь tests/results.txt файлд хадгалагдсан. Дахин ажиллуулахад тогтвортой байх ёстой шалгуур нь viol=0, max_eat ≤ N/2 болон "ALL TESTS PASSED" мөр юм.



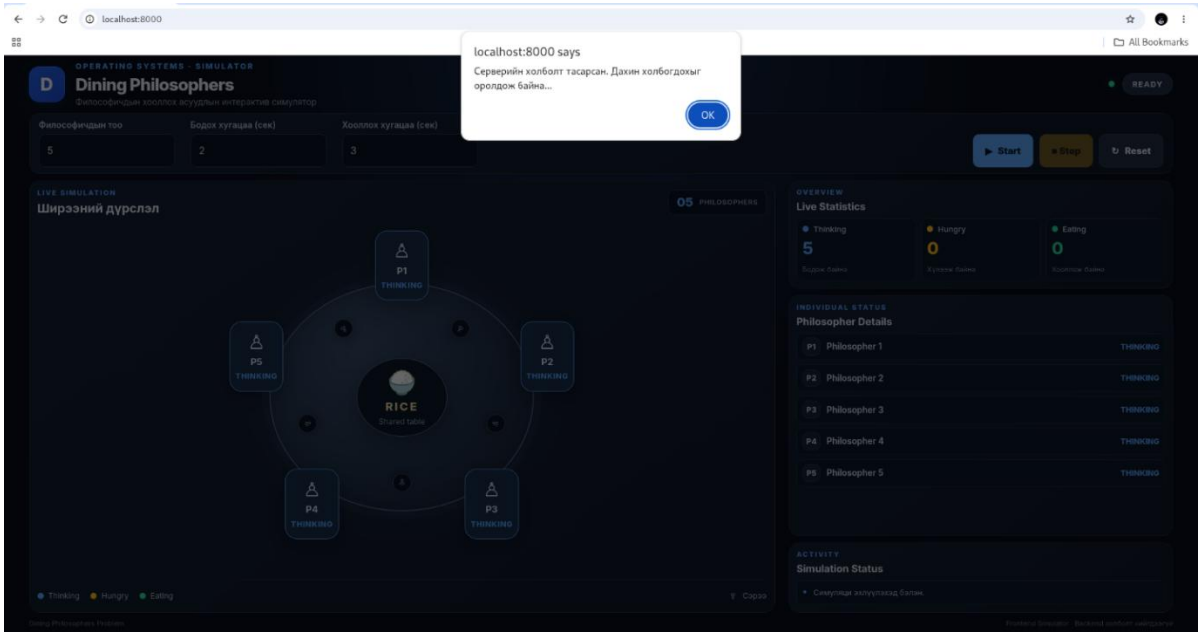
Зураг 3 N=10 үеийн UI дүрслэл

Зураг 3 нь философичдын тоог дээд хязгаар болох N=10 болгосон үеийн UI-г харуулна. Ширээн дээрх философич, сэрээний дүрс тоотой нь уялдан автоматаар өөрчлөгдөж, философич бүр (P1–P10) дугуй ширээг тойрон жигд байрласан байна. Энэ агшинд 4 философич бодож (P1, P3, P6, P8), 2 нь өлсөж (P5, P10), 4 нь хооллож байна (P2, P4, P7, P9). Хоолж буй философичдын хэн нь ч хоорондоо хөрш биш бөгөөд зэрэг хооллогчийн тоо N/2 = 5-аас хэтрээгүй. Мөн өлссөн P5 ба P10 нь хөрш нь (P4, P9) хоол идэж байгаа тул сэрээ авч чадалгүй хүлээж байгаа нь monitor шийдлийн зөв ажиллагааг харуулна. Баруун талын жагсаалт философичдын тоо ихсэхэд гүйлгэх (scroll) боломжтой болно.



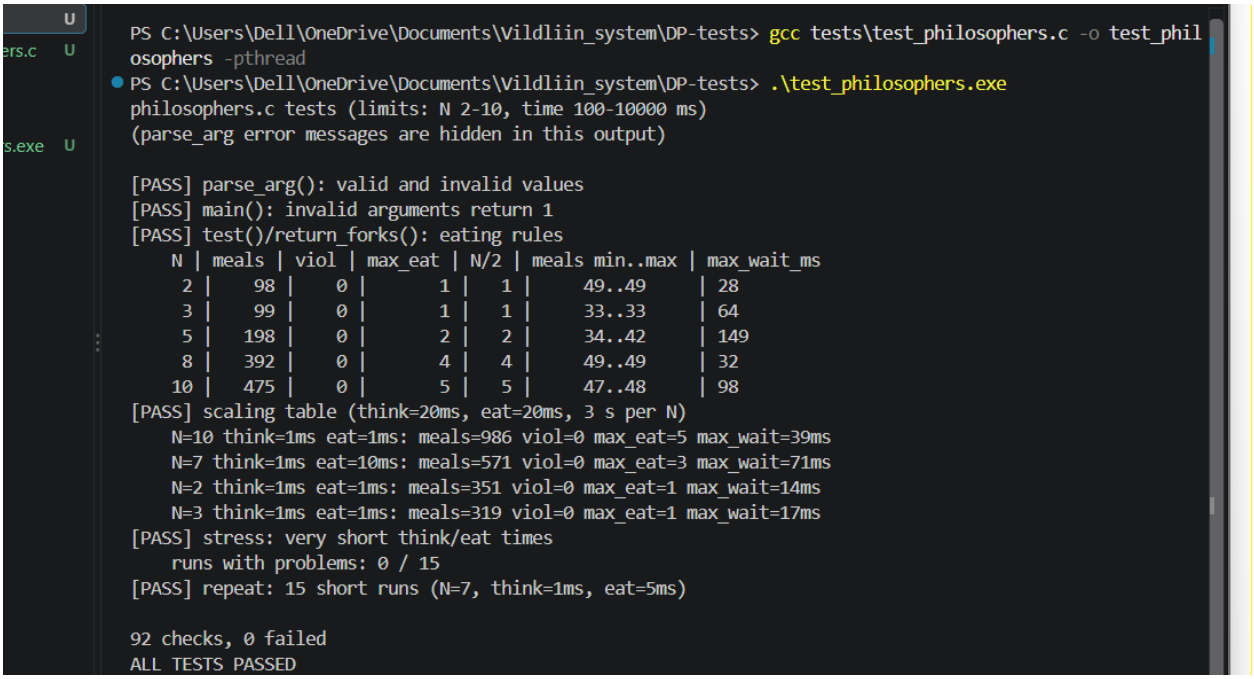
Зураг 4 Буруу оролтод UI-ийн 'ERROR' анхааруулга (жишээ нь N=-3)

Зураг 4 нь буруу оролтын шалгалтыг харуулна. "Философичдын тоо" талбарт хүчинтэй муж (2–10)-аас гадуурх -3 утгыг оруулан Start дарахад UI "Философичдын тоо 2–10 бүхэл тоо байх ёстой." гэсэн ойлгомжтой анхааруулга гаргаж, симуляцйг эхлүүлэхгүй. Ингэснээр буруу утга С программ хүртэл дамжихгүй бөгөөд ширээн дээрх дүрс болон төлөв (5 философич, бүгд Thinking) өөрчлөгдөхгүй хэвээр үлдэнэ. Сервер ба С программ ч мөн ижил мужийг дахин шалгадаг.



Зураг 5 server.py зогсоход UI-ийн 'ERROR' төлөв

Зураг 5 нь симуляц ажиллаж байх үед server.py-г зориудаар зогсоосон (Negative Testing) тохиолдлын UI-ийн хариуг харуулна. EventSource-ийн onerror үйл явдал ажиллаж, bridge-client.js "Серверийн холболт тасарсан. Дахин холбогдохыг оролдож байна..." гэсэн мэдэгдлийг хэрэглэгчид гаргана. Ингэснээр backend тасарсан үед UI гацаж, хэрэглэгч шалтгааныг мэдэхгүй үлдэхгүй, серверийг дахин асаахад холболт автоматаар сэргэнэ.



Зураг 6 Автомат тестийн терминал гаралт (92 checks, ALL TESTS PASSED)

Зураг 6 нь tests/test_philosophers.c автомат тестийг ажиллуулсан терминал гаралтыг харуулна. Тестийг gcc-ээр хөрвүүлж ажиллуулахад parse_arg() функцийн зөв ба буруу утгын шалгалт, main()-ийн буруу аргументад 1 кодоор гарах байдал, test()/return_forks()-ийн хооллох дүрэм бүгд [PASS] болсон. Хүснэгтэд N=2, 3, 5, 8, 10 бүрт viol=0 (хөрш хоёр зэрэг хооллосон тохиолдол байхгүй), max_eat нь N/2-оос хэтрээгүй, философич бүр хоол идсэн (meals min..max) болохыг харуулна. Стресс тест (Think=Eat=1 мс) болон N=7 дээр 15 удаа давтсан ажиллуулалтад асуудалтай ажиллуулалт 0/15 байв. Төгсгөлд "92 checks, 0 failed — ALL TESTS PASSED" гэж гарсан нь Deadlock болон зэрэгцээ хооллолтын зөрчил гараагүйг баталж байна.

10. ХЭЛЭЛЦҮҮЛЭГ

Хөгжүүлэлтийн болон туршилтын явцад гарсан гол бэрхшээл ба шийдлүүд:

1. **CORS болон порт зөрчил:** Анх Live Server (5500) ба Backend (8000) тусдаа байснаас CORS алдаа гарч байсныг Python Bridge серверт static HTML/JS файл сервердэх логик нэмснээр 8000 порт дээр нэгтгэн бүрэн шийдвэрлэсэн.
2. **Хязгаарлалт:** Философичдын тоо 2–10 (`MAX_N=10`, статик массив), Think/Eat хугацаа 100–10000 мс гэж хязгаарласан тул N=1 болон N>10 нөхцөл дэмжигдэхгүй (татгалзана). N-ийг өсгөхийн тулд массивын хэмжээг өөрчилж thread-ийн тоо, санах ойн ачааллыг хэмжих шаардлагатай.
3. **Starvation:** Monitor шийдэл Deadlock-ийг арилгадаг ч fairness-ийг онолын хувьд баталгаажуулдаггүй. Туршилтад философич бүрийн хооллолтын зөрүү бага (жишээ

нь N=5-д 34..42) бөгөөд хэн ч бүх хугацаанд өлсөж үлдээгүй боловч энэ нь баталгаа биш, хэмжилт юм.

4. **Номын шаардлагаас зөрүү:** Сурах бичиг Think/Eat хугацааг 1–3 секундийн санамсаргүй байхаар заасан бол манай C программ философич бүрт ижил, тогтмол (тохируулдаг) хугацаа ашигладаг. Санамсаргүй хугацаатай горим нэмж дахин хэмжих нь цаашдын ажил.
5. **Тохиргооны дамжуулалт:** `--pass-args` сонголтгүй үед UI-ийн Think/Eat болон N≠5 C программмд дамжихгүй. Үүнийг default болгох нь зүйтэй.
6. **Платформ ба процессын цэвэрлэгээ:** `stdbuf` нь Linux/WSL-д л байна. Ажиллаж байхад server.py-г зогсоовол C процесс үлдэх (orphan) тохиолдол ажиглагдсан тул зогсоох үед процессыг цэвэрлэх handler нэмэх шаардлагатай.

11. ДҮГНЭЛТ

Энэхүү бие даалтын ажлаар F.CSM302 Үйлдлийн систем хичээлийн хүрээнд сурах бичгийн Project 3 (Dining-Philosophers)-ийг POSIX mutex lock болон condition variable ашиглан C хэл дээр хэрэгжүүлж, Python SSE bridge болон вэб UI-тай холбон 'Dining Philosophers Simulator'-ийг бүтээж, туршилт болон QA ажиллагааг гүйцэтгэлээ. Автомат тестийн 92 шалгалт бүгд давж, N=2–10 хүрээнд, мөн 1 мс-ийн стресс горимд ч зэргэлдээ хоёр философич зэрэг хооллосон тохиолдол гараагүй (viol=0), систем гацаагүй нь нотлогдсон. Буруу оролтыг C, сервер, UI түвшинд барьж, backend тасалдлыг UI-д мэдээлдэг. Тогтмол Think/Eat хугацаа, N≤10, `--pass-args` шаардлага зэрэг хязгаарлалтыг 9-р бүлэгт тэмдэглэв.

12. АШИГЛАСАН МАТЕРИАЛ

[1] A. Silberschatz, P. B. Galvin, G. Gagne. Operating System Concepts. Wiley, 2018. 7-р бүлэг: Synchronization Examples (7.1.3, Programming Project 3 — The Dining-Philosophers Problem).
[2] The Linux man-pages project: pthreads(7), pthread_mutex_lock(3), pthread_cond_wait(3).
[3] Python Software Foundation. Python documentation: subprocess, http.server, argparse. https://docs.python.org/3/
[4] MDN Web Docs. Server-Sent Events, EventSource. https://developer.mozilla.org/docs/Web/API/EventSource
[5] Төслийн репозитори: https://github.com/Amina7088/Dining-Philosophers-Problem

13. ХАВСРАЛТ А. АЖЛЫН ТӨЛӨВЛӨГӨӨ

1. Төслийн ерөнхий мэдээлэл

Талбар	Мэдээлэл
Хуваарилалтын код	4-2-11
Төсөл	Dining-Philosophers-Problem
Repository	GitHub Link
Хэл / Сан / Орчин	C (POSIX Threads), Python 3 (http.server, subprocess), HTML5/CSS3/JavaScript (SSE), Linux Debian / WSL2
Үндсэн дүрслэл	Браузер дээрх интерактив дугуй ширээний дүрслэл (Philosophers, Forks, Real-time Status)

2. Багийн гишүүдийн ажлын хуваарилалт

Гишүүн	Хариуцах ажил	Commit / File
А.Аминхишиг (B242270011)	Программын үндсэн логик ба Синхрончлол	src/philosophers.c
Т.Буяндэлгэр (B222270052)	Интерактив UI хөгжүүлэлт (Frontend / Visualizer)	src/frontend/index.html src/frontend/script.js src/frontend/style.css
Д.Гантогтох (B242270139)	Backend & UI холболт (Integration)	src/bridge/server.py src/frontend/bridge-client.js
Б.Билэгбарьмид (B241940060)	Тест болон Алдааны менежмент (Testing & QA)	tests/test_philosophers.c tests/results.txt
А.Аминхишиг (B242270011)	Тайлан бичих (Report & GitHub Management)	Report.pdf README.md
Д.Гантогтох (B242270139)	Хэрэглэгчийн болон Operational заавар (Manual.pdf)	Manual.pdf

3. Туршилтын төлөвлөгөө ба хүлээгдэж буй үр дүн

Тест	Оролт / Нөхцөл	Хүлээгдэж буй үр дүн
1	Стандарт туршилт N = 5, Think = 2 сек, Eat = 3 сек	Систем гацалтгүй (Deadlock-гүй) ажиллаж, зэргэлдээ хоёр философич нэгэн зэрэг хооллохгүй (viol = 0). Төлвүүд UI дээр бодит цагт шинэчлэгдэнэ.
2	Захын утгын туршилт (Хамгийн бага ба хамгийн их N): N = 2 болон N = 10	Систем N=2 дээр хамгийн ихдээ 1 хооллогч, N=10 дээр хамгийн ихдээ 5 хооллогчтойгоор гацалтгүй, зөрчилгүй хэвийн ажиллана.
3	Буруу оролтын шалгалт (Negative testing): N = -3, N = 0, N = 20 эсвэл үсгэн утга (abc)	С программ, сервер болон UI түвшинд буруу утгыг татгалзаж, хэрэглэгчид ойлгомжтой алдааны анхааруулга (Error) харуулна.

4	Стресс тест (Ачаалалтай орчин): N = 10, Think = 1 мс, Eat = 1 мс	Хурдан шилжилтийн үед ч thread-үүд зөрчилдөхгүй, зэргэлдээ хоёр философич зэрэг хооллохгүй (viol = 0), санах ойн алдаа гарахгүй.
5	Сүлжээ ба Серверийн тасалдлын тест: Симуляц ажиллаж байхад server.py-г зориудаар зогсоох	Вэб UI дээр серверийн холболт тасарсныг барьж авснаар 'ERROR' төлвийг харуулж, сануулга мэдээлэл өгнө.